MEDIA QUERIES &
THE MOBILE-FIRST APPROACH

Beginner-friendly CSS notes with explanations, examples, and practical exercises

Learning goals

- Understand what responsive web design means.
- Learn how CSS media queries work and how to write breakpoints.
- Understand the mobile-first approach and why it is useful.
- Build a responsive webpage that adapts to phones, tablets, and desktops.

Prepared as classroom teaching notes

1. What is Responsive Web Design?

Responsive web design is a way of building websites so that the layout and content adjust to different screen sizes and devices. A responsive page should be usable on a small phone, a tablet, a laptop, and a large desktop monitor.

Why is it important?

• People visit websites using devices with different screen widths.

• Text, images, cards, and navigation should fit the screen without unnecessary horizontal scrolling.

• A good responsive layout improves readability, accessibility, and the overall user experience.

• It helps one website work across many devices instead of requiring a separate mobile website.

Responsive design usually combines

• **Flexible layouts:** use percentages, Flexbox, Grid, and flexible containers.

• **Flexible media:** make images fit their containers.

• **Media queries:** apply different CSS rules when conditions such as viewport width are met.

2. What is a Media Query?

A CSS media query lets you apply a group of CSS rules only when a condition is true. The condition may check the viewport width, orientation, or a user's preference such as reduced motion.

For example, you can show one card per row on a phone and three cards per row on a wider screen.

Basic syntax

```
@media (condition) {
  /* CSS rules apply when the condition is true */
}
```

Meaning: `@media` starts the media query. The condition checks whether the device or viewport matches a requirement. CSS inside the braces applies only when that requirement is met.

Example: change the page background on wider screens

```
body {
  background-color: lightyellow;
}

@media (min-width: 768px) {
  body {
    background-color: lightblue;
  }
}
```

On viewports narrower than 768 CSS pixels, the background stays light yellow. At 768 pixels and wider, it becomes light blue. The original rule remains the default, and the media query overrides it when its condition matches.

3. Common Media Query Conditions

A. min-width

`min-width` means the viewport width must be at least the stated size. It is often used in mobile-first CSS to add layout improvements as the screen gets wider.

```
@media (min-width: 600px) {
```

```
  /* applies at 600px and wider */
}
```

B. max-width

`max-width` means the viewport width must be no more than the stated size. It can be useful when adapting an existing desktop-first stylesheet, though a mobile-first workflow commonly uses `min-width`.

```
@media (max-width: 599px) {
  /* applies at 599px and narrower */
}
```

C. orientation

Checks whether the viewport is in portrait or landscape orientation.

```
@media (orientation: landscape) {
  /* applies when viewport is wider than it is tall */
}
```

D. prefers-reduced-motion

Respects a user's operating-system preference to reduce non-essential motion, such as animations.

```
@media (prefers-reduced-motion: reduce) {
  * {
    animation-duration: 0.01ms;
    animation-iteration-count: 1;
    scroll-behavior: auto;
  }
}
```

For real projects, test animation rules carefully and avoid using motion that is essential to understanding content.

4. What is a Breakpoint?

A breakpoint is a screen width where a layout needs to change to remain clear and usable. For example, a row of three cards may fit on a laptop but feel cramped on a phone, so the cards can stack vertically below a breakpoint.

There is no single set of breakpoints that every website must use. Choose breakpoints based on where your content starts to look cramped or poorly arranged, not only on the names of devices.

Example breakpoint plan

Example range	Possible layout
Small screens: under 600px	One card per row; compact spacing
Medium screens: 600px and wider	Two cards per row
Large screens: 900px and wider	Three cards per row; wider container

5. What is the Mobile-First Approach?

The mobile-first approach means writing the base CSS for small screens first, then using media queries to add or change styles for wider screens. You start with the simplest, narrow layout and progressively enhance it for larger viewports.

How it works

• Write the default CSS for a phone-sized screen.

• Make content readable and easy to use with a simple layout.

• At a wider breakpoint, use a media query to add columns, increase spacing, or change navigation.

• Test the design at several widths, including widths between your chosen breakpoints.

Mobile-first example

```
/* Base styles: phone first */
.container {
  width: 100%;
  padding: 16px;
}

.cards {
  display: grid;
  grid-template-columns: 1fr;
  gap: 16px;
}

/* Wider screens */
@media (min-width: 600px) {
  .container {
    max-width: 900px;
    margin: 0 auto;
  }

  .cards {
    grid-template-columns: repeat(2, 1fr);
  }
}

/* Large screens */
@media (min-width: 900px) {
  .cards {
    grid-template-columns: repeat(3, 1fr);
  }
}
```

On a phone, the cards appear in one column. From 600px, they appear in two columns. From 900px, they appear in three columns.

6. Mobile-First vs Desktop-First

Mobile-first	Desktop-first
Starts with styles for small screens.	Starts with styles for large screens.
Often uses `min-width` queries to add features as space increases.	Often uses `max-width` queries to simplify the layout on smaller screens.
Encourages prioritising essential content and touch-friendly controls.	Can be convenient when adapting an existing desktop design.
A strong default for many new responsive projects.	Still valid; the best approach depends on the project.

7. Set the Viewport in HTML

For responsive layouts to behave as expected on mobile browsers, include the viewport meta tag in the HTML `<head>`. Without it, some mobile browsers may use a wider virtual viewport and scale the page down.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Responsive Cards</title>
  <link rel="stylesheet" href="style.css">
```

```
</head>
<body>
  <!-- Page content goes here -->
</body>
</html>
```

The `width=device-width` tells the browser to use the device's CSS viewport width. `initial-scale=1.0` sets the initial zoom level.

8. Practical Example: Responsive Service Cards

Create two files in the same folder: **index.html** and **style.css**. Paste the code below into the matching files. Open index.html in a browser and resize the browser window to see the layout change.

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Our Services</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <header class="site-header">
    <h1>Our Services</h1>
    <p>Simple solutions for your needs</p>
  </header>

  <main class="container">
    <section class="cards">
      <article class="card">
        <h2>Web Design</h2>
        <p>Clean, user-friendly website layouts.</p>
      </article>
      <article class="card">
        <h2>Development</h2>
        <p>Responsive pages built for different screens.</p>
      </article>
      <article class="card">
        <h2>Support</h2>
        <p>Help keeping your website working well.</p>
      </article>
    </section>
  </main>
</body>
</html>
```

style.css

```
* {
  box-sizing: border-box;
}

body {
  margin: 0;
  font-family: Arial, sans-serif;
  line-height: 1.6;
  background: #f3f6f8;
  color: #203040;
}

.site-header {
  text-align: center;
  padding: 24px 16px;
  background: #16324f;
  color: white;
}
```

```
.container {
  width: 100%;
  padding: 16px;
  margin: 0 auto;
}

.cards {
  display: grid;
  grid-template-columns: 1fr;
  gap: 16px;
}

.card {
  background: white;
  padding: 20px;
  border-radius: 10px;
  box-shadow: 0 2px 8px rgba(0, 0, 0, 0.08);
}

@media (min-width: 600px) {
  .container {
    max-width: 900px;
    padding: 24px;
  }

  .cards {
    grid-template-columns: repeat(2, 1fr);
  }
}

@media (min-width: 900px) {
  .container {
    max-width: 1100px;
  }

  .cards {
    grid-template-columns: repeat(3, 1fr);
  }
}
```

Expected result

• Phone / narrow screen: the three service cards stack vertically.

• Medium screen (600px and wider): two cards appear side by side, with the remaining card on the next row.

• Large screen (900px and wider): all three cards fit in one row.

9. Common Mistakes and How to Fix Them

Forgetting the viewport meta tag: Add `<meta name="viewport" content="width=device-width, initial-scale=1.0">` inside the HTML head.

Putting every style inside media queries: Keep the small-screen default styles outside media queries; use queries only for changes needed at wider sizes.

Choosing breakpoints only for specific devices: Resize the layout and choose a breakpoint where the content actually needs more space.

Using fixed widths everywhere: Prefer flexible widths, max-width, Grid, Flexbox, and responsive images where appropriate.

Forgetting to test intermediate widths: Test narrow phones, wider phones, tablets, laptops, and widths between breakpoints.

Media query not working: Check the spelling, braces, units such as px, whether the CSS file is linked, and whether another rule overrides the property.

10. Useful Responsive CSS Tips

```
/* Images should not overflow their containers */
img {
  max-width: 100%;
  height: auto;
}

/* Keep content readable on wide screens */
.container {
  width: 100%;
  max-width: 1100px;
  margin: 0 auto;
  padding: 16px;
}

/* Use flexible columns */
.layout {
  display: grid;
  grid-template-columns: 1fr;
  gap: 16px;
}

@media (min-width: 800px) {
  .layout {
    grid-template-columns: 2fr 1fr;
  }
}
```

Remember: media queries are only one part of responsive design. Flexible layouts, readable text, images that scale, and good content structure matter too.

11. Class Practice

Task: Build a responsive webpage for a school, service, or small business.

Requirements:

• Create a heading and a short introductory paragraph.

• Add three cards with a title and description in each card.

• On small screens, show one card per row.

• At 600px and wider, show two columns.

• At 900px and wider, show three columns.

• Include the viewport meta tag and link an external CSS file.

Challenge

Add a navigation bar. On small screens, allow the links to wrap onto more than one line. On wider screens, display them in a neat horizontal row.

Quick questions

1. What is a media query?

2. What does min-width mean?

3. What is a breakpoint?

4. Explain the mobile-first approach in your own words.

5. Why is the viewport meta tag important?

6. What is one difference between mobile-first and desktop-first CSS?

12. Short Answer Key

1. A CSS feature that applies styles when specified conditions are met.

2. The viewport must be at least the stated width.

3. A width where the layout changes to stay usable and readable.

4. Write the base styles for small screens first, then enhance them for wider screens.

5. It helps the browser match the layout viewport to the device width.

6. Mobile-first usually starts with small-screen styles and adds rules with min-width; desktop-first starts large and often uses max-width to adapt downward.

Summary

Media queries let CSS respond to conditions such as viewport width. A breakpoint is the point where the layout needs to change. With a mobile-first approach, begin with a simple small-screen layout and add improvements for wider screens using min-width media queries. Combine this with Grid or Flexbox, flexible containers, responsive images, and testing at different widths.